IRP-TW-DT

Developer Implementation Guide

Inventory Routing Problem with Time Windows & Dynamic Traffic

Hanoi Last-Mile Logistics — Production Codebase

Field	Value
Project	IRP-TW-DT Research Implementation
Stack	Python 3.11+ · NumPy · HiGHS (via highspy)
Scope	Production-quality research codebase, not demo
Experiment matrix	4 scenarios × 3 scales × 5 seeds = 60 runs
Planning horizon T	7 days | m = 2/3/5 vehicles by scale
Document version	v1.0 — March 2026

0. Quick Orientation

Purpose: This document is the single source of truth for implementing the IRP-TW-DT solver. Sections 1–3 cover the mathematical contract. Sections 4–6 cover data, experiments, and analysis. Section 7 is the living tracklist. Read in order on first pass; use as reference after.

0.1 Problem in one sentence

A depot serves **n customers** over **T = 7 days** with **m homogeneous vehicles**. Decide **who gets delivery on which day, how much, and in what route order** to minimize: inventory holding cost + vehicle distance cost + time-dependent travel cost. Customers have time windows [e_i, l_i] and travel times depend on a 5-zone Hanoi congestion profile.

0.2 What makes this hard

- **Multi-period inventory coupling:** delivering today affects holding cost for days 2–7. Route decisions and inventory decisions cannot be separated.
- **Non-linear travel times:** τ_ij(w_ikt) is piecewise-nonlinear. There is no fixed distance matrix. Every fitness evaluation calls igp_travel_time().
- **TW + inventory hard constraints:** both must hold simultaneously. Infeasible instances must be caught at data generation, not at solve time.

> ⚠ **Key architectural decision — MINLP handling**
>
> The objective is formally a MINLP (not MILP). We handle this as follows:
>
> **HGA path:** compute τ_ij(w_ikt) directly via IGP on every evaluation — no linearization.
>
> **MILP lower-bound path:** precompute τ̂_ij for 3 fixed departure slots {6h, 9h, 19h}, substitute before building LP. Used only for n ≤ 10, T = 3 validation.

1. Data Model & Contracts

1.1 Instance schema

All distance units are km. All time units are decimal hours (8.5 = 08:30). Index 0 is always the depot.

```
@dataclass
class Instance:
    name: str               # e.g. "hanoi_n20_seed42"
    n: int                  # number of customers
    T: int                  # planning horizon in days (default 7)
    m: int                  # number of vehicles

    # Spatial — index 0 = depot
    coords: np.ndarray      # (n+1, 2)   GPS-like (x, y) in km
    dist:   np.ndarray      # (n+1, n+1) Euclidean distance matrix in km

    # Inventory per customer (length n)
    U:      np.ndarray      # max warehouse capacity
    L_min:  np.ndarray      # safety stock floor
    I0:     np.ndarray      # initial inventory at t=0
    demand: np.ndarray      # (n, T) deterministic daily demand
    h:      np.ndarray      # holding cost VND/unit/day

    # Time windows — same every day
    e: np.ndarray           # (n,) earliest service start (hours)
    l: np.ndarray           # (n,) latest service start (hours)
    s: np.ndarray           # (n,) service time in hours (default 0.25)

    # Costs
    c_d: float              # VND/km
    c_t: float              # VND/hour
    Q:   float              # vehicle capacity in units
```

1.2 Parameter values

Parameter	Value	Note
T	7 days	IRP literature standard

Parameter	Value	Note
m (n=20)	2	—
m (n=50)	3	—
m (n=100)	5	—
Q	500 units	—
s_i	0.25 h (15 min)	All customers, fixed
c_d	3,500 VND/km	Estimate — calibrate before final runs
c_t	74,000 VND/h	Midpoint of 68k–80k (JobsGO 2024)
Customer radius	≤ 15 km	From Hoan Kiem depot (Euclidean)
TW shift 1	[8h, 12h]	Morning — 50% of customers
TW shift 2	[14h, 18h]	Afternoon — 50% of customers
H	24 h	Full operational day
Big-M	24.0 (= H)	Tight and valid for all time constraints

1.3 Traffic speed profile (IGP model)

5-zone piecewise-constant speed, calibrated from TomTom Traffic Index 2025. Recalibrate before final experiments if GPS data is available.

Zone	Hours [a_s, a_{s+1})	Speed km/h	Traffic state
1	[0h, 6h)	27	Night — sparse
2	[6h, 9h)	15	Morning peak
3	[9h, 17h)	19	Business hours
4	[17h, 19h)	14	Evening peak — worst
5	[19h, 24h)	21	Evening recovery

FIFO guarantee (mandatory unit test)

For all $0 \le \tau_1 < \tau_2$ and all distances d: $\tau_1 + \tau(d, \tau_1) \le \tau_2 + \tau(d, \tau_2)$. Verify with 1,000 random (d, τ) pairs before any integration.

1.4 Solution schema

```
@dataclass
class Solution:
    instance: Instance
    Y: np.ndarray          # (n, T)    bool  — delivery allocation
    q: np.ndarray          # (n, m, T) float — delivery quantities
    routes: list           # routes[t][k] = list of customer indices
    w: np.ndarray          # (n+1, m, T) float — service start times; 0=depot
    cost_inventory: float
    cost_distance:  float
    cost_time:      float
    total_cost:     float
    feasible:       bool
    tw_violations:       int
    stockout_violations: int
    capacity_violations: int
```

2. Mathematical Model

2.1 Objective function

$$Z = \sum_t \sum_i h_i \cdot I_{it} + \sum_t \sum_k \sum_{(i,j)} c_d \cdot d_{ij} \cdot x_{ijkt} + \sum_t \sum_k \sum_{(i,j)} c_t \cdot \tau_{ij}(w_{ikt}) \cdot x_{ijkt}$$

Inventory holding | Distance cost (linear) | Time-dependent travel cost (non-linear)

2.2 Constraints

ID	Constraint	Formula	Notes
C1	Inventory balance	$I_{it} = I_{i,t-1} - d_{it} + \sum_k q_{ikt}$	Couples days together
C2	Inventory bounds	$L_{min,i} \le I_{it} \le U_i$	Safety stock floor + capacity cap
C3	Order-up-to-level	$q_{ikt} \le (U_i - I_{i,t-1}) \cdot z_{ikt}$	Can only fill empty space
C4	Vehicle capacity	$\sum_i q_{ikt} \le Q \quad \forall k,t$	Hard constraint
C5	Flow out	$\sum_j x_{ijkt} = z_{ikt} \quad \forall i \in N', k,t$	One outgoing arc if visited
C6	Flow in	$\sum_j x_{jikt} = z_{ikt} \quad \forall i \in N', k,t$	One incoming arc if visited
C7	Depot departure	$\sum_j x_{0jkt} \le 1 \quad \forall k,t$	Each vehicle departs at most once/day
C8	Time propagation	$w_{jkt} \ge w_{ikt} + s_i + \tau_{ij}(w_{ikt}) - M(1-x_{ijkt})$	Non-linear; w_{0kt} is free depot departure time
C9	TW lower bound	$e_i \cdot z_{ikt} \le w_{ikt}$	Not before e_i if visited
C10	TW upper bound	$w_{ikt} \le l_i \cdot z_{ikt} + M(1-z_{ikt})$	Not after l_i if visited; M releases when z=0
C11	Subtour elimination	$\sum_{\{i,j \in S\}} x_{ijkt} \le \lvert S \rvert - 1$	Use MTZ in MILP; not needed in HGA
C12	Integrality	$x,z \in \{0,1\}$; $I,q,w \ge 0$; $0 \le w_{0kt} \le H$	w_{0kt} is a free decision variable

C8 implementation note

C8 is where MINLP lives. In HGA: call igp_travel_time(dist[i,j], w_ikt) directly. In MILP: substitute $\hat{\tau}_{ij}$ precomputed for the chosen departure slot.

2.3 IGP travel time algorithm

```
# Speed zones as list of (start_h, end_h, speed_kmh)
ZONES = [(0,6,27),(6,9,15),(9,17,19),(17,19,14),(19,24,21)]

def igp_travel_time(distance_km: float, depart_h: float) -> float:
    """
    Compute travel time in hours for a given distance and departure time.
    Accumulates time zone by zone until distance is exhausted.
    FIFO-compliant by construction.
    """
    remaining_km = distance_km
    current_h = depart_h
    elapsed_h = 0.0
    for (z_start, z_end, speed) in ZONES:
        if current_h >= z_end - 1e-9:
            continue                                # not yet in this zone
        time_available = z_end - max(current_h, z_start)
        dist_in_zone = speed * time_available
        if dist_in_zone >= remaining_km - 1e-9:
            elapsed_h += remaining_km / speed
            return elapsed_h
        elapsed_h += time_available
        remaining_km -= dist_in_zone
        current_h = z_end
    return elapsed_h  # should not reach here if distance is within range

# ⚠  Always use 1e-9 epsilon in zone boundary comparisons.
# Test departures at exactly 6.0, 9.0, 17.0, 19.0 — off-by-one zone is the
# most common bug.
```

3. HGA Algorithm Design

3.1 Architecture

Component	Role	Complexity per call
Two-part Chromosome	Stores allocation matrix Y (n×T) + Giant Tour π (n)	O(n·T) per individual
TD-Split	DP decode: Y+π → concrete routes with costs	$O(n^2 \cdot T)$ per evaluation
Fitness function	Z + weighted penalty terms	Wraps TD-Split
Crossover	Uniform (Y) + OX (π)	O(n·T) per child
Mutation	Bit-flip (Y) + swap/inversion (π) + repair	O(n·T)
Repair	Enforce no-stockout by forcing Y_it=1 where needed	O(n·T) — always run after crossover/mutation
2-opt	Spatial route improvement per day	$O(n^2)$ per route
Time-Shift NS	Temporal: shift customer i from day t to t'	O(n·T) per individual — core contribution

3.2 Chromosome

```
@dataclass
class Chromosome:
    Y:  np.ndarray    # (n, T) dtype=bool   allocation matrix
    pi: np.ndarray    # (n,)   dtype=int    Giant Tour permutation
    fitness:  float = np.inf
    feasible: bool   = False

# Y[i, t] = True  →  customer i gets delivery on day t
# pi = [3, 7, 1, 0, ...]  →  relative visit order for routing
# Delivery quantity q_it = U_i - I_i,t-1  (order-up-to-level)
```

3.3 TD-Split decode

```
DEPARTURE_SLOTS = [6.0, 9.0, 19.0]
```

```
def decode(chrom: Chromosome, inst: Instance) -> Solution:
    total_cost = 0.0
    all_routes = []
    for t in range(inst.T):
        D_t = [i for i in chrom.pi if chrom.Y[i, t]]  # customers for day t,
ordered by π
        best_routes, best_cost = None, np.inf
        for dep in DEPARTURE_SLOTS:
            routes, cost = td_split(D_t, dep, inst)   # DP partition into vehicle
routes
            if cost < best_cost:
                best_routes, best_cost = routes, cost
        all_routes.append(best_routes)
        total_cost += best_cost
    return build_solution(chrom, all_routes, total_cost, inst)

def td_split(D_t, depart_h, inst):
    # V[j] = min_{i<=j} { V[i-1] + route_cost(D_t[i:j+1], depart_h) }
    # route_cost = c_d * distance + c_t * travel_time + waiting_time_cost
    # Rejects sub-route if load exceeds Q (continues DP, penalty applied later)
    ...
```

3.4 Fitness function

```
# Penalty weights — stockout is the hardest constraint
LAMBDA_STOCKOUT  = 1_000_000
LAMBDA_CAPACITY  = 100_000
LAMBDA_TW        = 10_000

def fitness(chrom: Chromosome, inst: Instance) -> float:
    sol = decode(chrom, inst)
    P_s = count_stockout_day_violations(sol, inst)   # days where I_it < L_min_i
    P_c = count_capacity_violations(sol, inst)       # vehicle-day pairs over Q
    P_w = count_tw_violations(sol, inst)             # customers served outside
[e_i, l_i]
    chrom.feasible = (P_s == 0 and P_c == 0 and P_w == 0)
    return sol.total_cost + LAMBDA_STOCKOUT*P_s + LAMBDA_CAPACITY*P_c +
LAMBDA_TW*P_w

# In tournament selection: feasible individuals ALWAYS beat infeasible.
# Do not rely solely on the numeric penalty to enforce this.
```

3.5 Evolutionary operators

Crossover

```
def crossover(p1: Chromosome, p2: Chromosome) -> Chromosome:
    mask = np.random.rand(n, T) < 0.5
    Y_child = np.where(mask, p1.Y, p2.Y)   # Uniform crossover on Y
    pi_child = ox_crossover(p1.pi, p2.pi)   # OX on Giant Tour
    child = Chromosome(Y=Y_child, pi=pi_child)
    repair(child, inst)  # mandatory after every crossover
    return child
```

Mutation

```
def mutate(chrom: Chromosome, p_flip=0.05, p_swap=0.10):
    chrom.Y ^= (np.random.rand(n, T) < p_flip)  # bit-flip on Y
    if np.random.rand() < 0.5:
        i, j = np.random.choice(n, 2, replace=False)  # swap
        chrom.pi[i], chrom.pi[j] = chrom.pi[j], chrom.pi[i]
    else:
        i, j = sorted(np.random.choice(n, 2, replace=False))
        chrom.pi[i:j+1] = chrom.pi[i:j+1][::-1]  # inversion
    repair(chrom, inst)  # mandatory after every mutation
```

Repair operator

```
def repair(chrom: Chromosome, inst: Instance) -> None:
    """
    Forward-simulate inventory. Force Y[i,t]=1 wherever stockout is predicted.
    Also verify no single-day vehicle load exceeds m*Q; if so,
    redistribute excess deliveries to adjacent days.
    Run after EVERY crossover and mutation — never skip.
    """
    I = inst.I0.copy()
    for t in range(inst.T):
        deliveries = compute_deliveries_ou(chrom.Y[:, t], I, inst)
        I = I - inst.demand[:, t] + deliveries
        violations = I < inst.L_min
```

```
        if violations.any():
            chrom.Y[violations, t] = True
            deliveries[violations] = inst.U[violations] - (I[violations] -
deliveries[violations])
            I[violations] = inst.U[violations]
```

3.6 Local search

2-opt (spatial)

```
# Applied to top 20% of population, after every generation.
# Key: when reversing segment [i..j], recompute ALL travel times from
# position i-1 forward — do NOT reuse cached τ (they depend on departure time).
# Reference: Visser & Spliet (2020) efficient TD-move evaluation.

def two_opt(route: list, depart_h: float, inst: Instance) -> list:
    improved = True
    while improved:
        improved = False
        for i in range(1, len(route)-1):
            for j in range(i+1, len(route)):
                delta = td_two_opt_delta(route, i, j, depart_h, inst)
                if delta < -1e-6:
                    route[i:j+1] = route[i:j+1][::-1]
                    improved = True
    return route
```

Time-Shift Neighborhood Search — core contribution

```
# Applied after 2-opt for top 20% of population.
# Optimizes the delivery schedule on the TIME AXIS.
# This is fundamentally different from 2-opt (which optimizes spatial routes).

def time_shift_ns(chrom: Chromosome, inst: Instance) -> None:
    for i in range(inst.n):
        for t in range(inst.T):
            if not chrom.Y[i, t]:
                continue
            for t_prime in range(inst.T):
```

```
                if t_prime == t:
                    continue
                if is_feasible_shift(chrom, i, t, t_prime, inst):
                    delta = cost_delta_shift(chrom, i, t, t_prime, inst)
                    if delta < -1e-6:
                        chrom.Y[i, t] = False
                        chrom.Y[i, t_prime] = True
                        break   # greedy first-improvement; restart outer loop

# is_feasible_shift: verify I_it >= L_min_i for all affected days after the move.
# cost_delta_shift: recompute Z only for days t and t_prime — not the full horizon.
```

4. Data Generation

4.1 Synthetic Hanoi dataset generation

Field	Distribution	Note
Customer coordinates	5 Gaussian clusters, r ≤ 15 km from depot	Inherit cluster structure from CVRPTW thesis
U_i	Uniform(80, 200) integer	Max warehouse capacity per customer
L_min_i	U_i · Uniform(0.10, 0.20)	Safety stock 10–20% of capacity
I_i0	Uniform(L_min_i, 0.6·U_i)	Initial stock at 30–60% of capacity
d_it	Lognormal(μ=log(U_i/7), σ=0.3)	≈14% capacity/day; matches T=7 horizon
h_i	Uniform(100, 500) VND/unit/day	Holding cost per unit
e_i, l_i	{[8,12], [14,18]} — uniform random 50/50	Same window every day of the horizon
s_i	0.25 h fixed	All customers

4.2 Single-visit feasibility check

Mandatory before saving any instance. Reject and regenerate (up to 100 retries) if any customer fails.

```
def check_single_visit_feasibility(inst: Instance) -> tuple[bool, int|None]:
    """
    Verify: a vehicle departing at 6h can reach every customer
    and start service before l_i.
    Worst case: zone 2 speed (15 km/h), max distance 15 km → arrive 7.0h → OK for
[8,12].
    """
    EARLIEST_DEPART = 6.0
    for i in range(inst.n):
        tt = igp_travel_time(inst.dist[0, i], EARLIEST_DEPART)
        arrive = EARLIEST_DEPART + tt
        service_start = max(arrive, inst.e[i])  # wait if early
        if service_start + inst.s[i] > inst.l[i]:
            return False, i
    return True, None
```

4.3 Experiment matrix

Scenario	Model	Speed	Delivery trigger	Role
P — Periodic	Heuristic	$\bar{v}$ = 18 km/h	Fixed every 3 days for all customers	Simple industry rule — easiest baseline
A — RMI	CVRPTW	$\bar{v}$ = 18 km/h	Next-day when $I_{it} \leq L_{min_i}$	Reactive industry baseline
B — IRP-TW-Static	IRP-TW	$\bar{v}$ = 18 km/h	HGA without Time-Shift NS	Isolates VMI benefit from DT benefit
C — IRP-TW-DT	IRP-TW-DT	5-zone Hanoi profile	HGA with Time-Shift NS	Proposed full model

60 runs total: 4 scenarios × 3 scales (n=20/50/100) × 5 seeds {42,123,456,789,1000}.

Scenario P note: does not use HGA. Assign all customers to days {1, 4, 7}, route with nearest-neighbor. Implementation time < 2 hours.

5. MILP Lower-Bound Validation

5.1 Purpose and scope

Scope: n ≤ 10, T = 3, fixed departure time only

MILP is not the main solver. It provides a lower bound to validate HGA solution quality. With n=20 and T=7, HiGHS will not converge in 10 minutes on standard hardware.

5.2 MILP formulation for HiGHS

```
# Step 1: precompute lookup table once per instance
SLOTS = [6.0, 9.0, 19.0]
tau_hat = {}
for i in range(n+1):
    for j in range(n+1):
        if i != j:
            for s, dep in enumerate(SLOTS):
                tau_hat[(i, j, s)] = igp_travel_time(dist[i,j], dep)

# Step 2: before building LP for day t, pick the slot with lowest
# estimated routing cost (greedy pre-selection).

# Step 3: build MILP with highspy
# Subtour elimination: use MTZ constraints (not exponential SECs)
# Big-M = 24.0 throughout
# Time limit: 600 seconds; MIP gap tolerance: 2%
```

5.3 Validation protocol

1. Run HGA on 10 small instances (n ≤ 10, T = 3), record best total_cost.
2. Run HiGHS on same instances (10-min time limit, 2% gap tolerance).
3. gap_i = (HGA_cost_i − MILP_bound_i) / MILP_bound_i × 100%
4. Report: mean gap ± std across 10 instances. Target: mean gap ≤ 15%.
5. If mean gap > 15%: inspect convergence curves, tune lambda penalties, recheck repair operator. Do not proceed to full 60-run experiment until gap target is met.

6. Experiment Protocol & Analysis

6.1 HGA hyperparameters

Parameter	Default	Search range
Population size P	50	30 – 100
Max generations G	200	100 – 500
Tournament size k	5	3 – 10
Elitism rate	10% (top 5)	Fixed
Local search rate	20% (top 10)	Fixed
Crossover probability	0.9	0.7 – 1.0
Mutation flip prob (Y bits)	0.05	0.02 – 0.10
Mutation swap/inversion prob (π)	0.10	0.05 – 0.20
Time limit T_max	300 s (NumPy vectorized)	180 – 600 s

6.2 KPIs per run

KPI	Unit	Purpose
total_cost	VND	Primary metric — objective Z
cost_inventory	VND	Holding cost component
cost_distance	VND	Distance cost component
cost_time	VND	Time-dependent travel cost — should decrease in Scenario C
avg_inventory_level	% of U_i	Trade-off indicator
total_distance_km	km	All vehicles across all days
tw_compliance_rate	%	% deliveries within time window — should be 100% in feasible solutions
n_deliveries	count	Total deliveries across T=7 days
cpu_time_s	seconds	Wall-clock time per run

6.3 Statistical analysis plan

- **P vs A:** Does reactive RMI outperform simple periodic delivery? Two-sample t-test.
- **A vs B:** VMI scheduling contribution (same static speed). t-test or Wilcoxon depending on Shapiro-Wilk normality test.
- **B vs C:** Pure dynamic traffic contribution. Wilcoxon signed-rank (paired, non-normal expected).
- **Trade-off scatter:** Plot (cost_inventory, cost_distance + cost_time) per instance, colored by scenario. Check whether C shifts the frontier vs B.
- **Convergence:** Best-fitness vs generation for n=50, seed=42. Verify convergence before G_max.

Key managerial hypothesis to test

"Hanoi congestion justifies earlier deliveries and higher inventory levels." Test: does Scenario C show significantly higher cost_inventory but lower cost_time vs Scenario B? This is the practical insight reviewers look for.

7. Implementation Tracklist

30 tasks across 10 weeks. Update **Status** as work progresses. **DoD** = Definition of Done — must be met before marking DONE.

ID	Module	Task & DoD	Week	Status	Notes
T01	Traffic	Implement igp_travel_time(d, τ). DoD: 1,000 random (d,τ) pairs pass FIFO check. Zone boundaries {6.0, 9.0, 17.0, 19.0} produce correct results.	W1	TODO	Priority 1
T02	Traffic	Precompute τ̂ lookup table for MILP (3 slots × all (i,j)). DoD: table serializable to JSON; values match igp_travel_time() to 1e-6.	W1	TODO	
T03	Env	Project structure, venv, pyproject.toml, pytest config. DoD: pytest runs 0 errors on empty test suite; logging configured.	W1	TODO	
T04	Data	Instance dataclass + generator for n=20. DoD: generated instance passes schema validation + single-visit feasibility check.	W2	TODO	
T05	Data	check_single_visit_feasibility(). DoD: correctly rejects 3 hand-crafted failing instances.	W2	TODO	
T06	Data	Standalone inventory forward simulator. DoD: simulate_inventory(Y, demand, I0) matches 3 manually computed examples.	W2	TODO	
T07	Split	td_split(D_t, inst, depart_h). DoD: routes never exceed Q; cost verified on 2-customer manual case.	W3	TODO	
T08	Split	decode(chrom, inst) with 3 departure slots. DoD: returns valid Solution with correct cost breakdown vs manual.	W3	TODO	
T09	Eval	fitness() with 3 penalty terms. DoD: infeasible chromosomes ALWAYS have higher fitness than any feasible one (100 random pairs).	W3	TODO	
T10	GA	Chromosome + random_init(inst, P). DoD: all P individuals decode without exception.	W4	TODO	
T11	GA	repair(chrom, inst). DoD: repaired chromosome has zero stockout violations per simulate_inventory().	W4	TODO	
T12	GA	tournament_selection(pop, k). DoD: feasible always selected over infeasible in 1,000 trials.	W4	TODO	

ID	Module	Task & DoD	Week	Status	Notes
T13	GA	crossover(p1, p2) — Uniform(Y) + OX(π). DoD: child passes repair; π is valid permutation post-OX.	W5	TODO	
T14	GA	mutate(chrom). DoD: mean fitness decreases over 50 generations on n=20, seed=42.	W5	TODO	
T15	GA	Full GA loop (init→eval→select→crossover→mutate→eval). DoD: convergence curve is monotone non-increasing best fitness.	W5	TODO	
T16	LS	two_opt(route, depart_h, inst). DoD: reduces route cost on ≥80% of random initial routes (n=20). Recomputes τ after each swap.	W6	TODO	Visser & Spliet move eval
T17	LS	time_shift_ns(chrom, inst). DoD: improves total_cost ≥3% vs no local search, 5 seeds, n=20.	W6	TODO	Core contribution — test carefully
T18	GA	Hyperparameter grid search: P ∈ {30,50,100} × G ∈ {100,200,500}, n=20. DoD: best config documented.	W6	TODO	
T19	MILP	Install highspy; MILP formulation for IRP-TW static, n≤10, T=3. DoD: HiGHS solves 5-customer instance in <60s.	W7	TODO	
T20	MILP	Validate HGA vs MILP on 10 small instances. DoD: mean gap ≤15% documented in table.	W7	TODO	Retest HGA if gap >15%
T21	Data	Generate all instances: n=50 (m=3) and n=100 (m=5), 5 seeds. DoD: all 30 instances pass feasibility, saved to disk.	W7	TODO	
T22	Exp	Periodic baseline (P): every-3-day policy + nearest-neighbor routing. DoD: runs all 15 instances in <5 min.	W8	TODO	
T23	Exp	RMI baseline (A): reactive CVRPTW, static speed. DoD: cost matches 2 manual examples.	W8	TODO	
T24	Exp	Run all 60 experiments. DoD: all 60 CSV result files saved; no crashes; CPU times logged.	W8	TODO	Budget ~6h compute
T25	Analysis	t-test (A vs B), Wilcoxon (B vs C). DoD: p-values + effect sizes (Cohen's d or rank-biserial r) reported.	W9	TODO	
T26	Analysis	Trade-off scatter: (cost_inv, cost_route) per instance by scenario. DoD: Pareto shift for C visible.	W9	TODO	

ID	Module	Task & DoD	Week	Status	Notes
T27	Analysis	Convergence curve for n=50, seed=42. DoD: HGA converges before T_max; Time-Shift contribution visible.	W9	TODO	
T28	Report	Results section: all 60 runs + statistical test summary. DoD: no missing data.	W10	TODO	
T29	Report	Discussion: answer 4 research questions with evidence. DoD: managerial insight stated explicitly.	W10	TODO	
T30	Report	Slides 8–12 slides. DoD: timed ≤15 min; all charts included.	W10	TODO	

How to use this tracklist

Set IN PROGRESS when you start. Set DONE only when the stated DoD is fully met — not before. Set BLOCKED if a dependency is unfinished (note the blocking task ID). T01 and T05 are the two hard dependencies: nothing else works without them.

8. Environment Setup & Known Risks

8.1 Dependencies

Package	Version	Purpose
numpy	≥ 1.26	Core numerical ops — vectorize everything
scipy	≥ 1.12	Lognormal sampling; Wilcoxon, Shapiro-Wilk, t-test
highspy	≥ 1.7	HiGHS MILP solver for validation
pandas	≥ 2.2	Result aggregation, CSV I/O
matplotlib	≥ 3.8	Convergence curves, scatter plots
pytest	≥ 8.0	Unit and integration tests
dataclasses-json	any	Instance serialization to/from JSON

8.2 Suggested project structure

```
irp_tw_dt/
├── core/
│   ├── traffic.py       # igp_travel_time, speed zones, FIFO test
│   ├── instance.py      # Instance dataclass, generator, feasibility check
│   ├── solution.py      # Solution dataclass, cost breakdown
│   └── constants.py     # LAMBDA_*, ZONES, SLOTS, default params
├── solver/
│   ├── chromosome.py    # Chromosome, random_init, repair
│   ├── decode.py        # td_split, decode
│   ├── fitness.py       # fitness(), violation counters
│   ├── operators.py     # crossover, mutate, tournament_selection
│   ├── local_search.py  # two_opt, time_shift_ns
│   └── hga.py           # main GA loop
├── baselines/
│   ├── periodic.py      # Scenario P
│   └── rmi.py           # Scenario A
├── milp/
│   └── highs_model.py   # MILP lower-bound for validation
├── experiments/
│   ├── run_all.py       # 60-run orchestrator
│   └── analyse.py       # statistical tests + plots
├── data/
```

```
│   ├── instances/       # generated .json files
│   └── results/         # per-run .csv output
└── tests/
    ├── test_traffic.py  # FIFO, zone boundary tests
    ├── test_instance.py # feasibility check
    ├── test_decode.py   # td_split correctness
    └── test_hga.py      # convergence smoke test
```

8.3 Performance targets

Operation	Target	Strategy
igp_travel_time (single)	< 1 µs	Pure Python, precomputed zone array — no optimization needed
decode(n=100, T=7)	< 50 ms	NumPy vectorized dist matrix; avoid Python loops in td_split
Full HGA run n=100	< 5 min	NumPy throughout; profile before optimizing
All 60 experiments	< 6 h	Sequential single machine; no parallelism required
MILP validation n=10,T=3	< 10 min	HiGHS defaults; increase gap tolerance to 2% if needed

8.4 Known risks & mitigations

Risk	Likelihood	Mitigation
MILP timeout for n=10	Medium	Reduce to n=8, T=3. Increase MIP gap to 2%.
HGA gap > 15%	Low–Medium	Increase G to 500. Verify repair operator. Check lambda penalties.
Time-Shift NS < 3% gain	Low	Verify departure time diversity in td_split. Check cost_delta_shift correctness.
Float errors at zone boundaries	High	Always use 1e-9 epsilon. Regression tests for departures at 6.0, 9.0, 17.0, 19.0.
Infeasible instances slipping through	Low	check_single_visit_feasibility is mandatory. 100 retries per instance.